

Activity: RPG loadout announcer

You are helping a game developer create a **very small proof of concept** for an RPG.

The goal is to:

- define shared types in a separate file
- use union types to restrict valid values
- import those types into your app
- compile TypeScript and run the emitted JavaScript

This activity is **types only**. There is no game logic, no combat, no stats.

Folder structure

```
src/  
  app.ts  
  types/  
    hero.ts
```

src/types/hero.ts

Create and export the following **types**:

- **HeroClass**
 - "Warrior" | "Mage" | "Ranger" | "Thief"
- **ArmorType**
 - "Mail" | "Plate" | "Leather" | "Cloth"
- **WeaponType**
 - "Mace" | "Staff" | "Bow" | "Dagger"

Then create and export a **Hero interface** with:

- **id**: number
- **name**: string
- **class**: HeroClass
- **armor**: ArmorType
- **weapon**: WeaponType

src/app.ts

1. Import the **Hero** type.
2. Create a collection (array) of at least **four heroes** with different loadouts.
3. Create a function:

- `announceHero(hero)`
 - accepts a `Hero`
 - logs a short description of the hero

Example output (your wording can vary):

```
Zara the Mage wears Cloth and wields a Staff.  
Borin the Warrior wears Plate and wields a Mace.
```

4. Loop over the heroes and announce each one.

Demo checklist

In `app.ts`:

1. Create at least **four heroes**
2. Use at least **three different classes**
3. Use at least **three different weapons**
4. Call `announceHero` inside a loop
5. Log all heroes to the console

Bonus (optional)

Add one new property to `Hero`:

- `level: number`

Then:

1. Create a function:
 - `levelUp(hero)`
 - increases the hero's level by `1`
2. Level up all heroes in a loop.
3. Announce them again, now including level.

Example:

```
Zara the Mage (lvl 2) wears Cloth and wields a Staff.
```

Run

Compile and run your app using your `npm start` script.

```
"scripts": {  
  "start": "npx tsc && node dist/app.js"  
}
```

Yep - add a small **Reflection** section at the bottom with guided prompts. Keep it short, concrete, and “notice the limitation” without asking them to implement anything.

Here’s a drop-in section you can append to the activity (same formatting style):

Reflection

These questions are here to help you notice what TypeScript **types** can and cannot do on their own.

1. Equipment restrictions

- Right now, any class can equip any armor/weapon.
- If the game dev says:
 - *“Mages can only wear Cloth”*
 - *“Warriors can’t use Staff”*
- Where would that rule live in your code?
 - In a type?
 - In a function?
 - Somewhere else?

2. Attack + defense idea

- Imagine we want to compute:
 - **attack** based on **weapon**
 - **defense** based on **armor**
- With your current union types, what do you *not* have yet that you would need?
 - (Hint: unions list allowed words, but they don’t contain values like “Bow = 7 attack”.)

3. What happens as rules grow

- If you start adding rules like:
 - class restrictions
 - weapon stats
 - armor stats
 - “special effects” (e.g. Staff gives +mana)
- What do you think happens to your code if everything stays as types + a few functions?
 - Does it stay simple, or does it start to get messy?